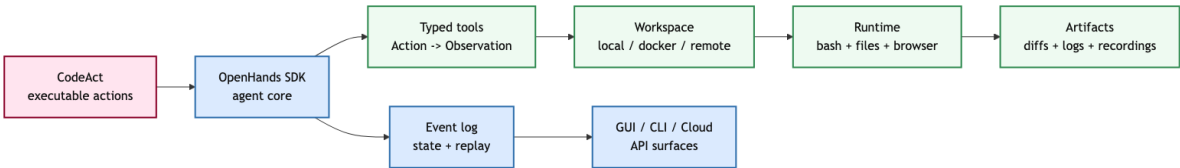


第 09 讲: OpenHands / CodeAct: Executable Action 的平台边界

Coding Agent Notes (June 2026)

1 核心图景

OpenHands 和 CodeAct 共同回答的是一个比“怎样写一个 agent loop”更工程化的问题: 当 coding agent 从实验 harness 走向长期可用的平台时, 系统到底多了什么?



CodeAct 先给出一个底层判断: 很多 agent task 的 action 不适合被压成很窄的 JSON 或自然语言命令, 因为真实任务需要临时变量、控制流、工具组合、异常处理和对中间结果的复用。OpenHands 则把这个可执行 action space 放进一个完整平台: typed event、conversation state、tool registry、workspace、runtime、UI/API、sandbox、security、persistence、browser recording、sub-agent delegation。

因此, 读 OpenHands 不应把重点放在“功能清单有多长”, 而应放在一个更集中的系统问题上: 当 action 可以执行任意代码, 平台必须用哪些边界来保存状态、隔离执行、审计风险, 并让用户理解 agent 做了什么。

这一讲的核心命题:
CodeAct 解决 action language 的表达力问题;
OpenHands 解决这些 action 在真实软件工程系统里如何运行、隔离、持久化、观察和集成
的问题。

这也是它和 mini-SWE-agent 的关键差别。mini-SWE-agent 问的是: 最小 shell loop 的上限在哪里? OpenHands 问的是: 如果能让很多用户、很多工具、很多会话、很多运行环境都能用同一个 agent core, 系统需要哪些稳定边界?

2 为什么需要平台

研究型 harness 通常只需要完成一次 rollout: 读任务、调用模型、执行动作、得到 patch、跑评测。平台型系统要承担更长的生命周期: 会话要能暂停和恢复, 轨迹要能重放, 工具要能注册和替换, 执行环境要能隔离, UI 要能实时展示, API 要能给 CLI、GUI、云服务和自动化调用。

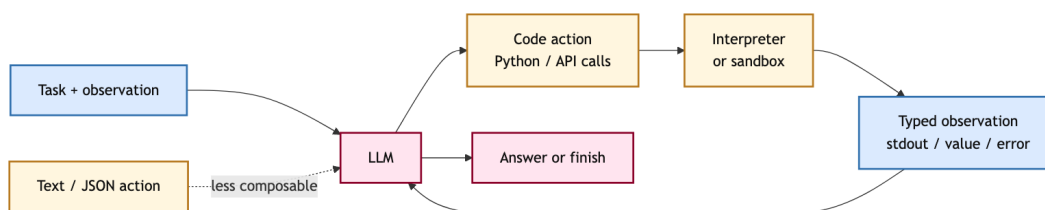
问题	Research harness	Platform system
一次任务	能完成 rollout	能管理多轮会话和中断
状态	messages 或 trajectory	append-only event log
执行	shell / docker enough	local、Docker、remote server
工具	写死在 prompt 或代码里	typed registry + schema
安全	依赖评测沙箱	risk + confirmation + audit
用户界面	日志即可	UI、CLI、API、replay

平台化的本质不是“功能堆叠”，而是把原本隐含在 harness 里的假设变成显式系统边界。比如在一个 benchmark 脚本里，状态丢失可能只意味着这一题失败；在平台里，状态丢失意味着用户无法恢复工作、UI 无法解释 agent 为什么做了某个修改、后续调试没有证据。

Insight. 从 harness 到 platform，核心变化是责任边界扩大了。模型能力仍然重要，但系统必须同时对状态、执行、权限、持久化、可观察性和集成负责。

3 CodeAct: action language

CodeAct 的出发点很直接：LLM 已经在代码上训练得很充分，那么 agent action 为什么不能直接使用代码？如果 action 是一段可执行 Python，模型可以把多个工具调用、数据清洗、分支判断和异常恢复写在同一段逻辑里，而不是被迫在每一步输出一个扁平 JSON。



文本 action 的优点是自然，缺点是解析不稳定。JSON tool call 的优点是结构清晰，缺点是每个动作往往很窄；复杂计划需要很多轮小动作，轮与轮之间的数据流要靠自然语言或外部状态维持。CodeAct 把 action 放回编程语言：变量可以保存中间结果，函数可以封装步骤，异常可以被捕获，包可以被导入，循环可以表达重复探索。

3.1 表达力从哪里来

CodeAct 的表达力不是来自 Python 语法本身，而是来自三个系统效果。第一，代码天然支持组合。模型可以在一个 action 里读取文件、调用 API、整理结果、再决定下一步输出。第二，代码天然支持中间状态。JSON action 常常要把中间值显式塞进下一轮 prompt；代码 action 可以把结果存为变量、文件或对象。第三，代码天然支持 self-debug。执行错误会以 traceback、stdout、stderr 或返回值进入 observation，模型可以据此修改后续 action。

论文和仓库给出的系统也反映了这个选择：CodeActAgent 包含 LLM serving、interaction interface 和 containerized JupyterKernelGateway code execution engine。也就是说，CodeAct 不是只提出一个 prompt 格式；它必须配套一个能安全执行代码、返回 observation、保存交互轨迹的运行环境。

CodeAct 的系统含义：

action representation 决定 agent 能表达哪些计划；

一旦 action 变成 executable code，平台就必须负责执行边界、错误反馈和安全封装。

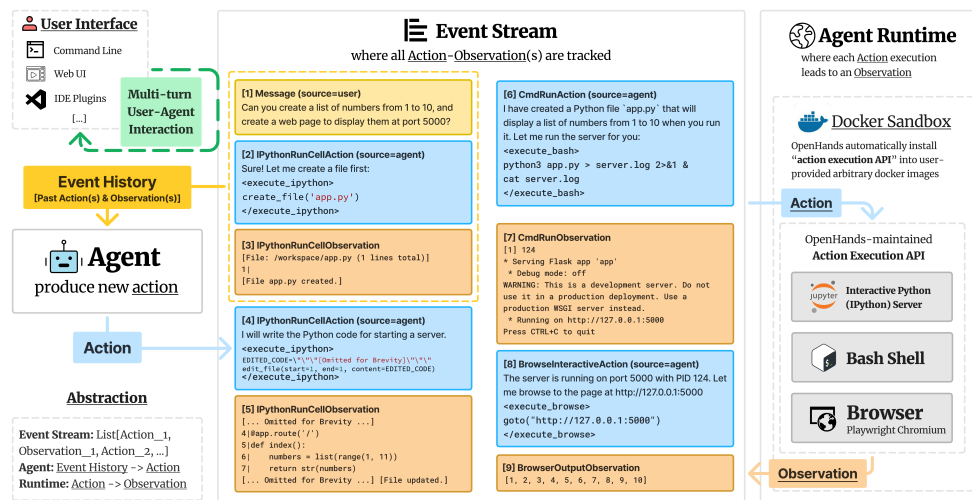
3.2 CodeActInstruct 和评测含义

CodeActInstruct 包含约 7k 个 multi-turn interaction trajectories。它的价值不只是数据量，而是让模型学习“写 action、观察执行结果、修正下一步”的交互格式。CodeAct 在 API-Bank 和 M³ToolEval 上比较 code、text、JSON action 形式，报告中强调 CodeAct 在研究设置中优于常见 text/JSON 表示。

这里应避免一个误读: CodeAct 并不是说所有工具都必须变成自由 Python。它更准确地说明了 action language 是 harness 设计的一部分。越强的 action language, 越需要更强的 runtime、权限、日志和测试来约束它。

4 OpenHands 论文架构

OpenHands 论文中的核心结构可以压缩成三件事: agent abstraction、event stream、runtime。agent abstraction 定义 agent 如何读取 state 并产生 action; event stream 保存动作和观察; runtime 执行动作并把结果变回 observation。



4.1 State 和 event stream

OpenHands 的 state 不只是 prompt 里的上下文。论文中 state 包含 chronological actions and observations、user interactions、累计 LLM cost、delegation metadata 和执行参数。event stream 是其中最重要的一部分, 因为每一次 action 和 observation 都会写入同一个时间序列。

这和普通 trajectory 的差别在于用途。trajectory 往往是事后分析或训练数据; event stream 是运行时系统的一部分。agent 读它来决定下一步, UI 读它来展示过程, 评测读它来判断行为, debugger 读它来定位错误, 持久化系统读它来恢复会话。

4.2 Actions 和 runtime

OpenHands 的 action space 受 CodeAct 启发, 但比单一 Python 更平台化。论文中明确出现的动作包括 bash command、IPython/Python cell、browser interaction、message、finish 和 delegation。Runtime 则提供 bash shell、Jupyter IPython server、Chromium browser, 以及一个运行在 Docker sandbox 里的 action execution API。

OpenHands paper 里的执行链:

agent 产生 action → event stream 记录 action → sandbox API 执行 action
→ runtime 返回 observation → observation 写回 event stream。

这个链条的关键是 sandbox API。它把“模型要执行的动作”和“真实操作系统、文件、浏览器”隔开。平台因此可以控制 workspace mount、Docker image、网络、browser state、返回格式和日志。

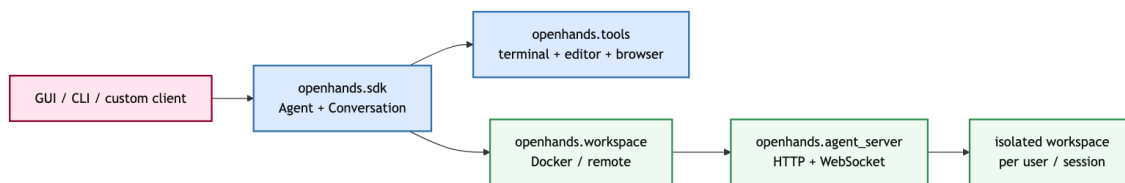
4.3 Skills 和 delegation

论文中的 AgentSkills 是一个很实用的设计：不是把所有能力都写成特殊 prompt，而是把常用能力作为 Python utilities 导入 IPython 环境。论文也给出选择标准：当原始 Python 不容易直接完成，或需要调用外部模型时，才值得加入 skill。

Delegation 则说明 OpenHands 的 platform boundary 不只服务单 agent。一个 generalist CodeActAgent 可以把 web browsing 任务委托给 specialized BrowsingAgent。这里最重要的不是“多 agent 一定更强”，而是平台必须能表达 subtask、sub-agent context、结果合并和错误传播。

5 当前 OpenHands SDK 架构

OpenHands 当前文档把系统进一步拆成四个包。SDK 管 agent 与 conversation，tools 管可调用动作，workspace 管执行环境，agent server 管远程执行入口。这比论文三组件更接近可维护平台的形态，也避免了把过长包名挤进同一行。

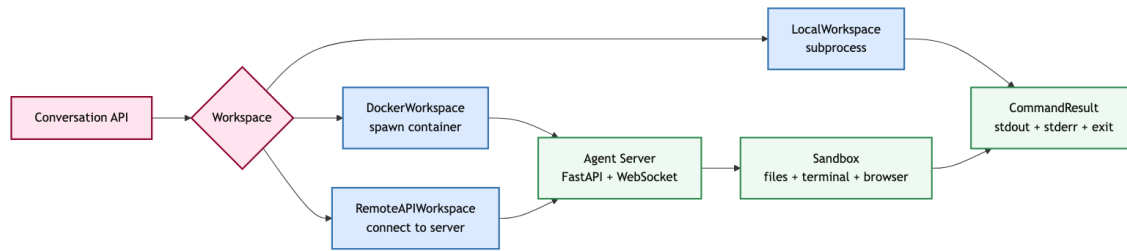


包	职责
openhands.sdk	Agent、Conversation、LLM、Event、Tool base、Workspace base、security、condenser
openhands.tools	terminal、file editor、grep、glob、browser、delegate、task tracker 等预置工具
openhands.workspace	DockerWorkspace、RemoteAPIWorkspace 等执行环境实现
openhands.agent_server	FastAPI + WebSocket 的远程 agent execution server

这个拆分有一个很强的系统设计含义：OpenHands 不再把 GUI、CLI、cloud、agent core、runtime 全部混在一个对象里。SDK 是 source of truth，界面层通过 SDK/API 调用 agent；workspace 决定执行环境；agent server 负责远程执行和多用户隔离。

5.1 Local mode 和 sandboxed mode

当前 docs 把部署分成两类。Local development mode 只需要 openhands-sdk 和 openhands-tools，一切在同一进程里运行，适合原型、可信环境和简单 CLI。Production 或 sandboxed mode 加入 openhands-workspace 和 openhands-agent-server，通过 DockerWorkspace 或 RemoteAPIWorkspace 把执行迁移到容器或远程 server。



这解释了 OpenHands V1 设计原则里的一个核心转向：sandboxing should be opt-in, not universal。强制 sandbox 对安全和复现有利，但会增加进程边界、状态同步、启动成本和 MCP/local tool 兼容问题。平台做成 workspace abstraction 后，同一套 agent code 可以在 LocalWorkspace、DockerWorkspace 和 RemoteAPIWorkspace 之间切换。

5.2 V0 到 V1：平台压力如何塑造架构

当前 OpenHands docs 对 V1 的设计原则非常有启发。它们不是抽象口号，而是从 V0 的平台压力中长出来的工程判断。

Optional isolation over mandatory sandboxing。

早期系统把每个 tool call 都放进 Docker sandbox，安全和复现性较强，但代价也很高：agent 和 sandbox 分属不同进程，状态容易分叉，启动和通信成本上升，多用户运行时更容易遇到资源和生命周期问题。V1 的选择是默认 local/simple，必要时再切到 Docker 或 remote。这个选择不是降低安全标准，而是承认 isolation 是一项可配置系统策略，不应该把所有用例都绑到最重的运行模式上。

One mutable state。

平台中最危险的 bug 往往不是模型答错，而是系统各处对“当前状态”的理解不一致。V1 把 agent、tools、LLM、config 这些组件做成更接近 stateless/immutable 的对象，把 conversation state 作为唯一可变状态。这样 pause、resume、fork、persistence、UI replay 都能围绕同一个状态源工作。

Clear boundary between agent core and applications。

如果 GUI、CLI、cloud、benchmark、integration 都直接改 agent core，系统很快会出现大量入口专属逻辑。V1 的做法是把 SDK 当成 agent source of truth，让应用层通过 API 和配置复用 SDK 对象。这个边界让研究实验、产品 UI 和部署系统可以独立演进。

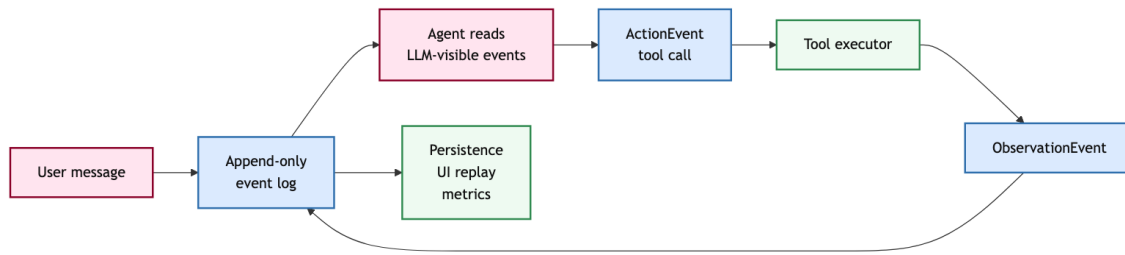
Composable components。

OpenHands 的 tools、skills、workspace、security、condenser、sub-agent 都被做成可组合组件。组合性不是为了“插件越多越好”，而是让平台能在不改核心 loop 的情况下替换能力：换模型、换工具集、换 workspace、换确认策略、换持久化方式。

Insight. V1 架构变化说明了一件事：真正的平台化不是添加更多按钮，而是让 agent core、执行环境、应用入口和安全策略彼此解耦。

6 Conversation 和 event stream

OpenHands SDK 的 Conversation 是生命周期管理器。它负责初始化 agent、接收 user message、运行 agent、协调 workspace、维护状态、触发 callback、持久化、暂停恢复、可视化和错误传播。它不是聊天记录的同义词，而是平台运行时的核心对象。



6.1 Event log 是 one source of truth

SDK docs 把 event system 描述成 immutable, type-safe, append-only log。这个设计解决的是平台系统里最容易出问题的部分：状态到底以谁为准？如果 agent object、UI object、runtime object、数据库 object 都各自维护一份状态，恢复会话和调试会变得非常脆弱。

OpenHands 的 event 类型分成两类。LLM-convertible events 会进入模型上下文，比如 MessageEvent、ActionEvent、ObservationEvent、SystemPromptEvent、AgentErrorEvent、UserRejectObservation、CondensationSummaryEvent。Internal events 不直接给模型看，比如 ConversationStateUpdateEvent、CondensationRequest、Condensation、PauseEvent。

事件类别	例子	作用
LLM-visible	message, action, observation	构成下一轮模型输入
Control	pause, condensation request	控制运行生命周期
State update	status, stats, metadata	同步非对话状态
Error	agent error, conversation error	区分可恢复工具错误和顶层失败

一个细节很重要：Event.source 和 LLM role 不是同一个东西。observation 可以来自 environment，但转成 LLM message 时是 tool role。framework 也可能注入来自 environment 的用户式反馈。平台需要同时记录“谁产生了事件”和“模型应该怎样读这个事件”。

6.2 为什么这比 messages 更强

mini-SWE-agent 的 linear messages 很清楚，适合作为最小设计。OpenHands 的 event log 则要承担更多职责：并行 tool call 需要通过 llm_response_id 分组，history condensation 需要明确哪些事件被遗忘和如何总结，pause/resume 需要保存中间状态，UI replay 需要知道每个 action 和 observation 的时间、来源和可视化形式。

Insight. 平台化状态管理的目标不是保存更多文本，而是让每个可见行为都有可追踪的事件来源、LLM 表示、执行结果和恢复路径。

7 Tools、workspace 和安全边界

OpenHands SDK 的 tool system 采用严格的 Action → Observation 模式。一个工具不是随便的 Python 函数，而是由 Pydantic Action、Pydantic Observation 和 ToolExecutor 组成。Action 定义输入 schema，Observation 定义结构化输出，Executor 执行业务逻辑。Agent 层再把它们包成 ActionEvent 和 ObservationEvent。

这个设计的好处是边界清楚。LLM 看到 schema，输出 tool call；框架验证 JSON 和 Pydantic 模型；工具执行后返回 typed observation；event log 保存 action 和 observation。工具本身不需要知道 conversation 如何持久化，也不需要知道 UI 如何展示。

7.1 Workspace 是执行环境接口

Workspace 抽象统一了本地进程、Docker container 和远程 API。LocalWorkspace 使用 subprocess；RemoteWorkspace 通过 HTTP 调 agent-server；DockerWorkspace 可以启动带 agent-server 的容器。命令执行返回 CommandResult，包括 stdout、stderr、exit code、timeout 和 duration。文件操作也统一成 upload/download 结果。

这看起来像普通工程抽象，但对 coding agent 很关键。agent 的 action 只有在某个 workspace 里才有含义：当前目录是什么、文件是否存在、命令是否超时、环境变量有哪些、容器是否能访问网络、修改是否会落到用户 filesystem。Workspace 把这些问题显式化。

7.2 Security 和 confirmation

平台不能只说“我们用了 Docker，所以安全”。OpenHands SDK 的 security docs 把安全拆成 risk assessment、confirmation policy、action validation 和 audit trail。LLMSecurityAnalyzer 可以让模型在生成 tool call 时同时给出 security risk；confirmation policy 决定 LOW、MEDIUM、HIGH、UNKNOWN 等风险是否需要用户确认。

这种机制并不完美，因为风险判断仍可能错。但它把权限问题放进 event/action 流程，而不是藏在 UI 外面。一个危险操作是否被允许、是否被拒绝、是否进入 observation，都能成为后续调试和审计的一部分。

平台安全的最低要求：

先定义 action 风险，再决定确认策略，最后把允许、拒绝和执行结果都写进事件流。

8 一个 repo 修复任务如何穿过平台

为了把这些组件连起来，可以追踪一个真实 repo 修复任务在 OpenHands 式平台里的路径。用户并不是直接把 prompt 发给模型，然后等待一个 patch；平台会把这个请求拆成一串可记录、可执行、可恢复的系统动作。

8.1 从 user message 到 event log

第一步，用户提交 issue 描述，例如“某个 parser 在空输入时抛出错误，请修复并补测试”。Conversation 把这条输入变成 MessageEvent，追加到 event log。此时 UI 可以显示用户消息，持久化系统可以保存会话，agent 还没有执行任何工具。

第二步，agent 读取 LLM-visible event，把 system prompt、用户消息、已有 action/observation、tool schema、security policy 转成模型输入。模型可能输出一个 grep 或 terminal action，例如搜索 parser 相关文件。SDK 把模型输出解析成 typed Action，验证参数，生成 ActionEvent。

第三步，tool executor 执行动作。这里 workspace 决定执行位置：LocalWorkspace 直接在本地 subprocess 里跑；DockerWorkspace 通过 agent-server 进入容器；RemoteAPIWorkspace 通过 HTTP 访问远程 server。命令执行结果被封装成 Observation，包括 stdout、stderr、exit code、timeout 等信息，再追加为 ObservationEvent。

一次工具调用的完整路径：

LLM tool call → Action validation → ActionEvent → ToolExecutor
→ Workspace execution → Observation → ObservationEvent → next LLM input。

8.2 从探索到 patch

接下来 agent 可能经历多轮搜索、阅读、编辑、测试。文件编辑工具会产生修改文件的 action; terminal 工具会运行 test; browser 工具可能打开文档或 issue 页面; delegate tool 可能把某个独立调查任务交给子代理。每一步都进入同一个 event log。

当 agent 写出修复并跑通测试，平台还需要保留证据：哪些文件被改过、执行过哪些测试、失败过哪些尝试、有没有用户拒绝某个危险动作、最终 summary 是否和 diff 一致。对于用户来说，这是可解释性；对于开发者来说，这是 debug 资料；对于训练和评测来说，这是高质量 trajectory。

8.3 为什么这一层不是可有可无

在单次 benchmark rollout 里，很多细节可以靠脚本绕过：失败就重跑，日志坏了就丢弃，环境脏了就重建。但平台不能这样工作。用户可能中途追加要求，可能暂停后第二天恢复，可能要审查 agent 为什么删除文件，可能要把同一个任务从 CLI 切到 GUI，可能要在云端容器里重放失败过程。

所以 OpenHands 的复杂度不是凭空产生的。它来自一个基本事实：一旦 coding agent 变成可交互软件，状态和证据就必须成为系统一等公民。

9 浏览器、技能与子代理

OpenHands 比简单 repo repair harness 更大，是因为它把软件工程 task 扩展到文件、terminal、browser、外部服务和多 agent 协作。Browser tool 不只是点击网页；docs 里还支持 browser session recording，把 DOM mutation、mouse movement、scrolling 等记录成 rrweb 可重放 JSON。这让 web task 的失败不再只是“模型说它点了”，而是有可检查证据。

Skills 则把可复用行为做成可加载知识。它们可以来自用户目录、repo 目录、组织目录或公共 registry。对平台来说，skills 的意义不是“prompt 更长”，而是把某类任务的操作规约、工具用法和项目约定从一次性对话中抽出来。这样 agent 在不同入口使用同一套知识。

Sub-agent delegation 提供另一类扩展：main agent 可以 spawn sub-agents，为它们分配独立任务，并在完成后合并结果。每个 sub-agent 有自己的 conversation context，但共享或受控访问同一 workspace。这个设计适合并行 research、分工检查、跨文件探索，但也会带来合并冲突和上下文一致性问题。

Insight. OpenHands 的扩展点可以分三类：tools 扩展动作能力，skills 扩展行为知识，sub-agents 扩展并行工作结构。三者都必须落回同一个 event/conversation/workspace 体系，才不会变成不可调试的插件堆。

10 局限与开放问题：平台化的代价

平台化带来能力，也带来系统成本。第一是边界成本。local、Docker、remote server 的行为不可能完全一样；文件路径、权限、网络、环境变量、进程生命周期都会变成 debug 对象。第二是序列化成本。事件、工具参数、observation、workspace 状态都要能被保存、传输、恢复，否则远程执行和 UI replay 就会不可靠。

第三是配置成本。模型 provider、workspace 类型、tool preset、security policy、skill loading、browser support、persistence directory、API key、container image 都可能影响结果。第四是安全成本。可执行 code action 越强，越需要 sandbox、confirmation、secret management 和 audit trail。第五是 UX 成本。用户需要知道什么时候 agent 正在运行、卡住、等待确认、修改了哪些文件、为什么失败。

收益	代价	设计检查
可恢复会话	状态序列化复杂	event log 是否能 replay
隔离执行	容器和网络开销	workspace 边界是否清楚
丰富工具	schema 和错误路径增多	observation 是否可行动
UI/API 集成	多入口一致性问题	SDK 是否是一致 source
多 agent	合并和协调成本	subtask 是否真正独立

因此，OpenHands 不是 mini-SWE-agent 的简单上位替代。mini 适合建立下限、做可解释 baseline、生成干净轨迹；OpenHands 适合需要长期会话、可视化、隔离、浏览器、多入口和平台扩展的场景。两者回答的问题不同。

11 四类系统的定位

把前面几讲的系统放在一起，OpenHands 的位置会更清楚。它不是“比所有 harness 更高级”的线性升级，而是占据一个不同的设计角落。

系统	最核心的问题	最适合的场景
mini-SWE-agent	最小 shell loop 是否足够强	baseline、trajectory、模型能力探测
SWE-agent	ACI 如何提升 repo repair	专用命令、简洁反馈、repair scaffold
Agentless	少自治是否更稳定	定位、采样、验证可分解的任务
OpenHands	平台如何承载长期 agent	UI/API、runtime、browser、多用户和扩展

这张表的关键不是排名，而是设计变量不同。mini-SWE-agent 把变量压到最少，因此最适合做“如果只给 shell，模型能走多远”的实验。SWE-agent 把变量集中在 interface design 上，因此最适合说明 action vocabulary 和 feedback format 对同一个模型的影响。Agentless 把策略外化成 pipeline，因此最适合说明有些任务不需要完全自治也能高效解决。OpenHands 则把变量扩展到平台边界，因此最适合讨论长期运行、用户协作、状态恢复、浏览器和远程执行。

11.1 什么时候 full platform 是必要的

当任务需要跨越多个 interaction surface 时，OpenHands 这类平台很有意义。比如一个 agent 要读 GitHub issue、修改 repo、跑测试、打开浏览器检查 UI、把过程展示给用户、等待确认、恢复会话、最后生成 summary。此时单一 shell loop 仍然能做一部分工作，但很难自然地承载 browser state、用户确认、UI replay、远程隔离和多入口一致性。

当系统要服务多个用户时，平台层几乎不可避免。多用户意味着 workspace 要隔离，credentials 要分离，conversation 要持久化，server 要能认证，tool execution 要能审计，event stream 要能被不同客户端订阅。此时 agent harness 已经变成 distributed application 的一部分。

11.2 什么时候 full platform 反而过重

如果任务只是离线跑一批 SWE-bench instances，目标是快速比较模型或收集干净轨迹，full platform 可能过重。额外的 server、container、UI、event schema 和安全策略会让实验变量变多，也让失败原因更难归因。此时 mini-SWE-agent 或专用 benchmark harness 更容易给出清晰信号。

如果任务有很强的固定结构，比如先定位、再采样多个 patch、再用测试筛选，Agentless 风格 pipeline 可能比开放式平台更经济。它牺牲一部分自治自由度，换来更强的可控性和候选选择能力。

Insight. 选择系统形态时，不要问“哪个最强”，要问“当前任务的主要不确定性在哪里”。不确定性在模型能力，就用最小 loop；在接口设计，就研究 ACI；在搜索和验证，就用 pipeline；在长期使用和集成，就需要平台。

12 如何读 OpenHands 成绩

OpenHands 论文展示了软件工程、web interaction 和 miscellaneous tasks 上的结果。读这些成绩时，重点不应该只是单个 benchmark 分数，而是同一个平台如何支持不同任务形态。SWE-bench 需要 repo、test、patch；web task 需要 browser action 和页面 observation；data analysis 或 general task 需要 Python/IPython 和文件系统。平台价值正体现在这些任务可以共享 agent abstraction、event stream 和 runtime。

这也解释了为什么 OpenHands paper 里的比较表会强调 GUI、standardized tool library、built-in sandbox and code execution、built-in browser、multi-agent collaboration、human-AI collaboration、AgentHub、evaluation framework、agent quality control。它不是在说每一项都直接提高 patch success rate，而是在定义一个通用 agent platform 的工程面。

读平台型 agent 的正确问题：

不是只问“分数多高”，还要问状态是否可恢复、动作是否可审计、执行是否隔离、工具是否可扩展、失败是否可解释、入口是否一致。

13 平台层带来的额外成本

OpenHands 的价值不是“把工具堆得更多”，而是把 executable action 放进一个可管理的平台里。但平台层本身也会制造成本：状态要同步，日志要持久化，权限要收口，容器要启动，浏览器要复现，UI 要能解释 agent 正在做什么。忽略这些成本，就会把平台误读成纯粹的能力增强。

平台能力	买到的东西	同时引入的成本
Event stream	可回放、可审计、可恢复	日志体积、隐私、压缩策略
Workspace abstraction	local / Docker / remote 可切换	状态同步和路径语义差异
Browser runtime	支持真实网页任务	页面 nondeterminism 和录制成本
Tool registry	工具可扩展、接口统一	工具权限和 schema 维护
Agent server	多用户远程执行	隔离、队列、资源回收
UI / CLI / cloud	多入口复用同一内核	产品状态和底层状态一致性

因此，平台型 agent 的设计问题不是“要不要 sandbox”，而是每一种执行能力是否有对应的状态、权限和证据闭环。一个 browser action 如果不能被记录，就难以 debug；一个 file edit 如果不能关联 diff，就难以 review；一个 delegated subtask 如果没有上下文边界，就难以判断失败责任。

13.1 平台边界的最小合同

如果把 OpenHands 抽象成通用平台合同，每个动作至少应该交代五件事：谁发起、在哪个工作区执行、需要什么权限、产生什么观察、如何在事件流中被恢复或回放。缺少任意一项，短 demo 仍可能成功，但长任务和多用户运行会变得不可维护。

合同字段	对 agent 的意义	对平台的意义
actor	区分 user、agent、sub-agent	支持审计和授权
workspace	明确动作作用域	支持 local / remote 切换
permission	知道哪些动作需要确认	防止权限隐式扩散
observation	让下一步可推理	统一 UI、日志和评测
replay key	能复现关键步骤	支持 resume、debug、rollback

这也是 CodeAct 和 OpenHands 的连接点。CodeAct 让 action language 更有表达力；OpenHands 则让这种表达力在真实系统里有边界、有记录、有恢复路径。前者解决“模型怎么表达动作”，后者解决“系统怎么承受这些动作”。

Insight. 平台不是把 agent 变聪明的魔法层。平台的真正作用，是让强 action language 的副作用可观察、可隔离、可恢复。没有这些合同，executable code action 只是在更大的空间里更快地产生不可控状态。

14 最终 takeaway

CodeAct 和 OpenHands 可以合成一句话：如果 action 变成 executable code，agent 的表达力会上升；如果要把这种表达力交给真实用户和真实代码库，平台层必须同时管理状态、执行、权限、观察和集成。

对系统设计来说，OpenHands 最有价值的地方不是“它工具多”，而是它把 coding agent 重新组织成可维护的边界：SDK core 管 agent 和 conversation，tools 提供 typed action/observation，workspace 管执行环境，agent-server 提供远程隔离，UI/CLI/cloud 复用同一个内核。

这给后续构建任何 coding agent 系统一个判断标准：先用最小 loop 确认可行性，再只在必要处增加平台层。增加平台层的理由必须足够具体：需要 durability、isolation、observability、human collaboration、browser interaction、multi-user execution 或 production integration。否则复杂度会先吞掉可解释性。

参考资料

- [1] [OpenHands](#). open platform architecture
- [2] [OpenHands SDK docs](#). conversation, events, tools, workspace
- [3] [Software Agent SDK](#). current SDK source
- [4] [OpenHands repository](#). GUI, cloud, enterprise code
- [5] [CodeAct](#). executable code actions
- [6] [CodeAct repository](#). data, executor, evaluation